ig - SYNTAX ERROR

Striver

a2z

DSA SHEET

- "495"

By - SYNTAX ERROR

k

Step 15: Binary Tree

The PDF created by *Abhishek Rathor (Instagram:SYNTAX ERROR)*

PROBLEM:1

Problem Statement:
Longest Substring Without Repeating Characters
Given a string, find the length of the longest substring without repeating characters.

Java Code:

```
java
CopyEdit
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Set<Character> set = new HashSet<>();
        int left = 0, right = 0, maxLen = 0;

        while (right < s.length()) {
            if (!set.contains(s.charAt(right))) {
                set.add(s.charAt(right));
                maxLen = Math.max(maxLen, right - left + 1);
                right++;
            } else {
                set.remove(s.charAt(left));
                left++;
            }
        }

        return maxLen;
    }
}
```

PROBLEM:2

Problem Statement:
Max Consecutive Ones III
Given a binary array and an integer k, find the maximum number of consecutive 1s in the array if you can flip at most k 0s.

Java Code:

```
java
CopyEdit
class Solution {
```

```
    public int longestOnes(int[] nums, int k) {
        int left = 0, right = 0, zeroCount = 0;

        while (right < nums.length) {
            if (nums[right] == 0) zeroCount++;
            if (zeroCount > k) {
                if (nums[left] == 0) zeroCount--;
                left++;
            }
            right++;
        }

        return right - left;
    }
}
```

PROBLEM:3

Problem Statement:
Fruit Into Baskets
You are given an array representing fruits. Return the length of the longest subarray with at most 2 distinct integers.

Java Code:

```
java
CopyEdit
class Solution {
    public int totalFruit(int[] fruits) {
        Map<Integer, Integer> map = new HashMap<>();
        int left = 0, maxLen = 0;

        for (int right = 0; right < fruits.length; right++) {
            map.put(fruits[right], map.getOrDefault(fruits[right], 0) + 1);

            while (map.size() > 2) {
                map.put(fruits[left], map.get(fruits[left]) - 1);
                if (map.get(fruits[left]) == 0) map.remove(fruits[left]);
                left++;
            }

            maxLen = Math.max(maxLen, right - left + 1);
        }

        return maxLen;
    }
}
```

PROBLEM:4

Problem Statement:
Longest Repeating Character Replacement

k

Given a string with only uppercase letters, return the length of the longest substring that can be obtained after replacing at most k characters.

Java Code:

```
java
CopyEdit
class Solution {
    public int characterReplacement(String s, int k) {
        int[] count = new int[26];
        int maxFreq = 0, left = 0, res = 0;

        for (int right = 0; right < s.length(); right++) {
            count[s.charAt(right) - 'A']++;
            maxFreq = Math.max(maxFreq, count[s.charAt(right) - 'A']);

            while ((right - left + 1) - maxFreq > k) {
                count[s.charAt(left) - 'A']--;
                left++;
            }

            res = Math.max(res, right - left + 1);
        }

        return res;
    }
}
```

PROBLEM:5

Problem Statement:
Binary Subarray With Sum
Given a binary array `nums` and an integer `goal`, return the number of non-empty subarrays with a sum equal to goal.

Java Code:

```
java
CopyEdit
class Solution {
    public int numSubarraysWithSum(int[] nums, int goal) {
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0, 1);
        int sum = 0, res = 0;

        for (int num : nums) {
            sum += num;
            res += map.getOrDefault(sum - goal, 0);
            map.put(sum, map.getOrDefault(sum, 0) + 1);
        }
```

```
        return res;
    }
}
```

PROBLEM:6

Problem Statement:
Count Number of Nice Subarrays
Given an array of integers and an integer k, return the number of nice subarrays (subarrays with exactly k odd numbers).

Java Code:

```
java
CopyEdit
class Solution {
    public int numberOfSubarrays(int[] nums, int k) {
        for (int i = 0; i < nums.length; i++)
            nums[i] = nums[i] % 2;

        return atMost(nums, k) - atMost(nums, k - 1);
    }

    private int atMost(int[] nums, int k) {
        int left = 0, res = 0;
        for (int right = 0; right < nums.length; right++) {
            k -= nums[right];
            while (k < 0) k += nums[left++];
            res += right - left + 1;
        }
        return res;
    }
}
```

PROBLEM:7

Problem Statement:
Number of Substrings Containing All Three Characters
Given a string `s` consisting only of characters 'a', 'b' and 'c'. Return the number of substrings containing at least one occurrence of each character.

Java Code:

```
java
CopyEdit
class Solution {
    public int numberOfSubstrings(String s) {
        int[] count = new int[3];
        int res = 0, left = 0;
```

```
        for (int right = 0; right < s.length(); right++) {
            count[s.charAt(right) - 'a']++;

            while (count[0] > 0 && count[1] > 0 && count[2] > 0) {
                res += s.length() - right;
                count[s.charAt(left++) - 'a']--;
            }
        }

        return res;
    }
}
```

PROBLEM:8

Problem Statement:
Maximum Points You Can Obtain from Cards
You have an array of card points. You can take k cards from either the beginning or end. Return the maximum score possible.

Java Code:

```
java
CopyEdit
class Solution {
    public int maxScore(int[] cardPoints, int k) {
        int total = 0;
        for (int i = 0; i < k; i++) total += cardPoints[i];

        int maxScore = total, n = cardPoints.length;
        for (int i = 0; i < k; i++) {
            total += cardPoints[n - 1 - i] - cardPoints[k - 1 - i];
            maxScore = Math.max(maxScore, total);
        }

        return maxScore;
    }
}
```

PROBLEM:9

Problem Statement:
Longest Substring with At Most K Distinct Characters
Given a string `s` and an integer `k`, return the length of the longest substring that contains at most `k` distinct characters.

Java Code:

```
java
CopyEdit
class Solution {
    public int lengthOfLongestSubstringKDistinct(String s, int k) {
        if (k == 0 || s.length() == 0) return 0;

        Map<Character, Integer> map = new HashMap<>();
        int left = 0, maxLen = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            map.put(c, map.getOrDefault(c, 0) + 1);

            while (map.size() > k) {
                char leftChar = s.charAt(left);
                map.put(leftChar, map.get(leftChar) - 1);
                if (map.get(leftChar) == 0) {
                    map.remove(leftChar);
                }
                left++;
            }

            maxLen = Math.max(maxLen, right - left + 1);
        }

        return maxLen;
    }
}
```

PROBLEM:10

Problem Statement:
Subarray with K Different Integers
Given an integer array `nums` and an integer `k`, return the number of good subarrays of `nums` where there are exactly `k` different integers.

Java Code:

```
java
CopyEdit
class Solution {
    public int subarraysWithKDistinct(int[] nums, int k) {
        return atMost(nums, k) - atMost(nums, k - 1);
    }

    private int atMost(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        int left = 0, res = 0;

        for (int right = 0; right < nums.length; right++) {
            map.put(nums[right], map.getOrDefault(nums[right], 0) + 1);

            while (map.size() > k) {
                map.put(nums[left], map.get(nums[left]) - 1);
                if (map.get(nums[left]) == 0) {
```

```
                    map.remove(nums[left]);
                }
                left++;
            }

            res += right - left + 1;
        }

        return res;
    }
}
```

PROBLEM:11

Problem Statement:
Minimum Window Substring
Given two strings `s` and `t`, return the minimum window in `s` which will contain all the characters in `t`. If no such window exists, return an empty string.

Java Code:

```
java
CopyEdit
class Solution {
    public String minWindow(String s, String t) {
        if (s.length() < t.length()) return "";

        Map<Character, Integer> map = new HashMap<>();
        for (char c : t.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }

        int left = 0, minLen = Integer.MAX_VALUE, count = 0, start = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (map.containsKey(c)) {
                map.put(c, map.get(c) - 1);
                if (map.get(c) >= 0) count++;
            }

            while (count == t.length()) {
                if (right - left + 1 < minLen) {
                    minLen = right - left + 1;
                    start = left;
                }

                char lc = s.charAt(left++);
                if (map.containsKey(lc)) {
                    map.put(lc, map.get(lc) + 1);
                    if (map.get(lc) > 0) count--;
                }
            }
        }
```

```
        return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start
+ minLen);
    }
}
```

PROBLEM:12

Problem Statement:
Minimum Window Subsequence
Given two strings `S` and `T`, find the minimum window in `S` which will contain `T` in sequence (not necessarily contiguous).

Java Code:

```
java
CopyEdit
class Solution {
    public String minWindow(String S, String T) {
        int sLen = S.length(), tLen = T.length();
        int minLen = Integer.MAX_VALUE, start = -1;

        for (int i = 0; i < sLen; i++) {
            if (S.charAt(i) == T.charAt(0)) {
                int sIdx = i, tIdx = 0;
                while (sIdx < sLen && tIdx < tLen) {
                    if (S.charAt(sIdx) == T.charAt(tIdx)) tIdx++;
                    sIdx++;
                }
                if (tIdx == tLen) {
                    int end = sIdx - 1;
                    sIdx = end;
                    tIdx = tLen - 1;
                    while (sIdx >= i) {
                        if (S.charAt(sIdx) == T.charAt(tIdx)) tIdx--;
                        if (tIdx < 0) break;
                        sIdx--;
                    }
                    if (end - sIdx + 1 < minLen) {
                        minLen = end - sIdx + 1;
                        start = sIdx;
                    }
                }
            }
        }

        return start == -1 ? "" : S.substring(start, start + minLen);
    }
}
```

PROBLEM:13

Problem Statement:
Introduction to Priority Queues using Binary Heaps
Implement a basic min-heap and max-heap using a PriorityQueue in Java.

Java Code:

```
java
CopyEdit
import java.util.*;

public class PriorityQueueIntro {
    public static void main(String[] args) {
        // Min Heap
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        minHeap.add(3);
        minHeap.add(1);
        minHeap.add(2);
        System.out.println("Min Heap: " + minHeap.poll()); // 1

        // Max Heap
        PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Collections.reverseOrder());
        maxHeap.add(3);
        maxHeap.add(1);
        maxHeap.add(2);
        System.out.println("Max Heap: " + maxHeap.poll()); // 3
    }
}
```

PROBLEM:14

Problem Statement:
Min Heap and Max Heap Implementation from Scratch
Implement min-heap and max-heap manually using an array.

Java Code (Min-Heap skeleton):

```
java
CopyEdit
class MinHeap {
    int[] heap;
    int size;

    public MinHeap(int capacity) {
        heap = new int[capacity];
        size = 0;
    }

    void insert(int val) {
        heap[size] = val;
        int i = size;
```

```
        size++;
        while (i > 0 && heap[i] < heap[(i - 1) / 2]) {
            int temp = heap[i];
            heap[i] = heap[(i - 1) / 2];
            heap[(i - 1) / 2] = temp;
            i = (i - 1) / 2;
        }
    }

    int extractMin() {
        if (size == 0) return -1;
        int min = heap[0];
        heap[0] = heap[size - 1];
        size--;
        heapify(0);
        return min;
    }

    void heapify(int i) {
        int smallest = i;
        int left = 2 * i + 1, right = 2 * i + 2;
        if (left < size && heap[left] < heap[smallest]) smallest = left;
        if (right < size && heap[right] < heap[smallest]) smallest = right;
        if (smallest != i) {
            int temp = heap[i];
            heap[i] = heap[smallest];
            heap[smallest] = temp;
            heapify(smallest);
        }
    }
}
```

PROBLEM:15

Problem Statement:
Check if an Array Represents a Min-Heap
Return true if the array satisfies min-heap property.

Java Code:

```
java
CopyEdit
class Solution {
    public boolean isMinHeap(int[] arr) {
        int n = arr.length;
        for (int i = 0; i <= (n - 2) / 2; i++) {
            if (arr[i] > arr[2 * i + 1]) return false;
            if (2 * i + 2 < n && arr[i] > arr[2 * i + 2]) return false;
        }
        return true;
    }
}
```

PROBLEM:16

Problem Statement:
Convert Min Heap to Max Heap
Given a min-heap represented as an array, convert it to a max-heap.

Java Code:

```
java
CopyEdit
class Solution {
    public void convertMinToMaxHeap(int[] arr) {
        int n = arr.length;
        for (int i = n / 2 - 1; i >= 0; i--) {
            maxHeapify(arr, n, i);
        }
    }

    void maxHeapify(int[] arr, int n, int i) {
        int largest = i;
        int l = 2 * i + 1;
        int r = 2 * i + 2;

        if (l < n && arr[l] > arr[largest]) largest = l;
        if (r < n && arr[r] > arr[largest]) largest = r;

        if (largest != i) {
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;
            maxHeapify(arr, n, largest);
        }
    }
}
```

PROBLEM:17

Problem Statement:
Topo Sort (DFS Based)
Perform Topological Sort on a Directed Acyclic Graph (DAG) using DFS.

Java Code:

```
java
CopyEdit
class Solution {
    public int[] topoSort(int V, ArrayList<ArrayList<Integer>> adj) {
        boolean[] visited = new boolean[V];
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < V; i++) {
```

```
            if (!visited[i])
                dfs(i, adj, visited, stack);
        }

        int[] result = new int[V];
        int index = 0;
        while (!stack.isEmpty())
            result[index++] = stack.pop();

        return result;
    }

    private void dfs(int node, ArrayList<ArrayList<Integer>> adj, boolean[]
visited, Stack<Integer> stack) {
        visited[node] = true;
        for (int nei : adj.get(node)) {
            if (!visited[nei])
                dfs(nei, adj, visited, stack);
        }
        stack.push(node);
    }
}
```

PROBLEM:18

Problem Statement:
Kahn's Algorithm (BFS Based Topo Sort)
Perform Topological Sort on a DAG using BFS (Kahn's algorithm).

Java Code:

```
java
CopyEdit
class Solution {
    public int[] kahnTopoSort(int V, ArrayList<ArrayList<Integer>> adj) {
        int[] indegree = new int[V];
        for (int i = 0; i < V; i++) {
            for (int nei : adj.get(i))
                indegree[nei]++;
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < V; i++)
            if (indegree[i] == 0)
                queue.add(i);

        int[] topo = new int[V];
        int index = 0;
        while (!queue.isEmpty()) {
            int node = queue.poll();
            topo[index++] = node;

            for (int nei : adj.get(node)) {
                indegree[nei]--;
                if (indegree[nei] == 0)
```

```
                    queue.add(nei);
            }
        }

        return topo;
    }
}
```

PROBLEM:19

Problem Statement:
Cycle Detection in Directed Graph (BFS)
Detect cycle in a directed graph using BFS and Kahn's algorithm.

Java Code:

```
java
CopyEdit
class Solution {
    public boolean hasCycle(int V, ArrayList<ArrayList<Integer>> adj) {
        int[] indegree = new int[V];
        for (int i = 0; i < V; i++) {
            for (int nei : adj.get(i))
                indegree[nei]++;
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < V; i++)
            if (indegree[i] == 0)
                queue.add(i);

        int count = 0;
        while (!queue.isEmpty()) {
            int node = queue.poll();
            count++;

            for (int nei : adj.get(node)) {
                indegree[nei]--;
                if (indegree[nei] == 0)
                    queue.add(nei);
            }
        }

        return count != V;
    }
}
```

PROBLEM:20

Problem Statement:
Course Schedule I

Determine if it's possible to finish all courses given prerequisites (cycle detection in directed graph).

Java Code:

```
java
CopyEdit
class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < numCourses; i++)
            adj.add(new ArrayList<>());

        int[] indegree = new int[numCourses];
        for (int[] pre : prerequisites) {
            adj.get(pre[1]).add(pre[0]);
            indegree[pre[0]]++;
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < numCourses; i++)
            if (indegree[i] == 0)
                queue.add(i);

        int count = 0;
        while (!queue.isEmpty()) {
            int node = queue.poll();
            count++;
            for (int nei : adj.get(node)) {
                indegree[nei]--;
                if (indegree[nei] == 0)
                    queue.add(nei);
            }
        }

        return count == numCourses;
    }
}
```

PROBLEM:21

Problem Statement:
Course Schedule II
Return the course order if it's possible to finish all courses.

Java Code:

```
java
CopyEdit
class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List<List<Integer>> adj = new ArrayList<>();
```

```
        for (int i = 0; i < numCourses; i++)
            adj.add(new ArrayList<>());

        int[] indegree = new int[numCourses];
        for (int[] pre : prerequisites) {
            adj.get(pre[1]).add(pre[0]);
            indegree[pre[0]]++;
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < numCourses; i++)
            if (indegree[i] == 0)
                queue.add(i);

        int[] result = new int[numCourses];
        int index = 0;

        while (!queue.isEmpty()) {
            int node = queue.poll();
            result[index++] = node;

            for (int nei : adj.get(node)) {
                indegree[nei]--;
                if (indegree[nei] == 0)
                    queue.add(nei);
            }
        }

        return index == numCourses ? result : new int[0];
    }
}
```

PROBLEM:22

Problem Statement:
Find Eventual Safe States
Return all nodes that are eventually safe in a directed graph (no cycle reachable from them).

Java Code:

```
java
CopyEdit
class Solution {
    public List<Integer> eventualSafeNodes(int[][] graph) {
        int n = graph.length;
        int[] indegree = new int[n];
        List<List<Integer>> reverseGraph = new ArrayList<>();

        for (int i = 0; i < n; i++)
            reverseGraph.add(new ArrayList<>());

        for (int i = 0; i < n; i++) {
            for (int v : graph[i]) {
                reverseGraph.get(v).add(i);
                indegree[i]++;
```

```
            }
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < n; i++)
            if (indegree[i] == 0)
                queue.add(i);

        boolean[] safe = new boolean[n];

        while (!queue.isEmpty()) {
            int node = queue.poll();
            safe[node] = true;
            for (int prev : reverseGraph.get(node)) {
                indegree[prev]--;
                if (indegree[prev] == 0)
                    queue.add(prev);
            }
        }

        List<Integer> result = new ArrayList<>();
        for (int i = 0; i < n; i++)
            if (safe[i])
                result.add(i);

        return result;
    }
}
```

PROBLEM:23

Problem Statement:
Alien Dictionary
Given a sorted dictionary of alien language, find the order of characters.

Java Code:

```
java
CopyEdit
class Solution {
    public String alienOrder(String[] words) {
        Map<Character, Set<Character>> graph = new HashMap<>();
        Map<Character, Integer> indegree = new HashMap<>();

        for (String word : words)
            for (char c : word.toCharArray())
                indegree.put(c, 0);

        for (int i = 0; i < words.length - 1; i++) {
            String w1 = words[i], w2 = words[i + 1];
            int minLen = Math.min(w1.length(), w2.length());
            for (int j = 0; j < minLen; j++) {
                char c1 = w1.charAt(j), c2 = w2.charAt(j);
                if (c1 != c2) {
                    if (!graph.containsKey(c1))
```

```
                    graph.put(c1, new HashSet<>());
                if (graph.get(c1).add(c2))
                    indegree.put(c2, indegree.get(c2) + 1);
                break;
            }
        }
    }

    Queue<Character> queue = new LinkedList<>();
    for (char c : indegree.keySet())
        if (indegree.get(c) == 0)
            queue.add(c);

    StringBuilder sb = new StringBuilder();
    while (!queue.isEmpty()) {
        char c = queue.poll();
        sb.append(c);
        if (graph.containsKey(c)) {
            for (char nei : graph.get(c)) {
                indegree.put(nei, indegree.get(nei) - 1);
                if (indegree.get(nei) == 0)
                    queue.add(nei);
            }
        }
    }

    return sb.length() == indegree.size() ? sb.toString() : "";
  }
}
```

PROBLEM:24

Problem Statement:
Shortest Path in Undirected Graph with Unit Weights
You are given an undirected graph. Find the shortest path from the source node to all other nodes using BFS (as all edges have weight 1).

Java Code:

```
java
CopyEdit
class Solution {
    public int[] shortestPath(int N, int M, int[][] edges, int src) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < N; i++) adj.add(new ArrayList<>());
        for (int[] edge : edges) {
            adj.get(edge[0]).add(edge[1]);
            adj.get(edge[1]).add(edge[0]);
        }

        int[] dist = new int[N];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        Queue<Integer> queue = new LinkedList<>();
        queue.add(src);
```

```
        while (!queue.isEmpty()) {
            int node = queue.poll();
            for (int nei : adj.get(node)) {
                if (dist[node] + 1 < dist[nei]) {
                    dist[nei] = dist[node] + 1;
                    queue.add(nei);
                }
            }
        }

        return dist;
    }
}
```

PROBLEM:25

Problem Statement:
Dijkstra's Algorithm
Find the shortest path from a source to all other nodes when all edges have non-negative weights.

Java Code:

```
java
CopyEdit
class Solution {
    static class Pair {
        int dist, node;
        Pair(int d, int n) {
            dist = d;
            node = n;
        }
    }

    public int[] dijkstra(int V, List<List<List<Integer>>> adj, int S) {
        PriorityQueue<Pair> pq = new PriorityQueue<>((a, b) -> a.dist -
b.dist);
        int[] dist = new int[V];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[S] = 0;
        pq.add(new Pair(0, S));

        while (!pq.isEmpty()) {
            Pair p = pq.poll();
            int d = p.dist, u = p.node;
            for (List<Integer> nei : adj.get(u)) {
                int v = nei.get(0), wt = nei.get(1);
                if (d + wt < dist[v]) {
                    dist[v] = d + wt;
                    pq.add(new Pair(dist[v], v));
                }
            }
        }
```

```
        return dist;
    }
}
```

PROBLEM:26

Problem Statement:
Bellman Ford Algorithm
Find shortest paths from a source in a graph that may have negative weights. Detect negative cycles.

Java Code:

```
java
CopyEdit
class Solution {
    public int[] bellmanFord(int V, int[][] edges, int S) {
        int[] dist = new int[V];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[S] = 0;

        for (int i = 0; i < V - 1; i++) {
            for (int[] edge : edges) {
                int u = edge[0], v = edge[1], wt = edge[2];
                if (dist[u] != Integer.MAX_VALUE && dist[u] + wt < dist[v])
{
                    dist[v] = dist[u] + wt;
                }
            }
        }

        for (int[] edge : edges) {
            int u = edge[0], v = edge[1], wt = edge[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + wt < dist[v]) {
                return new int[]{-1}; // Negative weight cycle
            }
        }

        return dist;
    }
}
```

PROBLEM:27

Problem Statement:
Floyd-Warshall Algorithm
Find shortest distances between every pair of vertices in a graph (all pairs shortest path).

Java Code:

k

```
java
CopyEdit
class Solution {
    public void floydWarshall(int[][] dist) {
        int V = dist.length;
        for (int k = 0; k < V; k++) {
            for (int i = 0; i < V; i++) {
                for (int j = 0; j < V; j++) {
                    if (dist[i][k] != 1e9 && dist[k][j] != 1e9)
                        dist[i][j] = Math.min(dist[i][j], dist[i][k] +
dist[k][j]);
                }
            }
        }
    }
}
```

PROBLEM:28

Problem Statement:
Minimum Spanning Tree (Kruskal's Algorithm)
Find a minimum spanning tree using union-find and sorting edges by weight.

Java Code:

```
java
CopyEdit
class DisjointSet {
    int[] parent, rank;

    DisjointSet(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++)
            parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }

    void union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return;

        if (rank[px] < rank[py])
            parent[px] = py;
        else if (rank[py] < rank[px])
            parent[py] = px;
        else {
            parent[py] = px;
            rank[px]++;
        }
```

```
    }
}

class Solution {
    public int kruskal(int[][] edges, int V) {
        Arrays.sort(edges, Comparator.comparingInt(a -> a[2]));
        DisjointSet ds = new DisjointSet(V);
        int cost = 0;

        for (int[] edge : edges) {
            int u = edge[0], v = edge[1], wt = edge[2];
            if (ds.find(u) != ds.find(v)) {
                cost += wt;
                ds.union(u, v);
            }
        }

        return cost;
    }
}
```

PROBLEM:29

Shortest Path in Undirected Graph with Unit Weights
Level: Hard

Problem Statement:
Given an undirected graph with unit weights (i.e., all edge weights are 1), find the shortest distance from the source to all nodes using BFS.

Java Code:

```
java
CopyEdit
public int[] shortestPath(int N, int M, int[][] edges, int src) {
    ArrayList<ArrayList<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < N; i++) adj.add(new ArrayList<>());

    for (int[] edge : edges) {
        adj.get(edge[0]).add(edge[1]);
        adj.get(edge[1]).add(edge[0]);
    }

    int[] dist = new int[N];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    Queue<Integer> q = new LinkedList<>();
    q.add(src);

    while (!q.isEmpty()) {
        int node = q.poll();
        for (int neighbor : adj.get(node)) {
            if (dist[node] + 1 < dist[neighbor]) {
                dist[neighbor] = dist[node] + 1;
                q.add(neighbor);
            }
        }
```

k

```
    }
    return dist;
}
```

PROBLEM:30

Shortest Path in DAG
Level: Hard

Problem Statement:
Given a Directed Acyclic Graph (DAG), find the shortest path from a source vertex to all other vertices.

Java Code:

```
java
CopyEdit
public int[] shortestPath(int N, int[][] edges, int src) {
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < N; i++) adj.add(new ArrayList<>());
    for (int[] edge : edges) {
        adj.get(edge[0]).add(new int[]{edge[1], edge[2]});
    }

    int[] dist = new int[N];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    Stack<Integer> stack = new Stack<>();
    boolean[] visited = new boolean[N];

    for (int i = 0; i < N; i++) {
        if (!visited[i]) topologicalSort(i, visited, stack, adj);
    }

    while (!stack.isEmpty()) {
        int u = stack.pop();
        if (dist[u] != Integer.MAX_VALUE) {
            for (int[] v : adj.get(u)) {
                if (dist[u] + v[1] < dist[v[0]]) {
                    dist[v[0]] = dist[u] + v[1];
                }
            }
        }
    }

    return dist;
}

private void topologicalSort(int node, boolean[] visited, Stack<Integer>
stack, List<List<int[]>> adj) {
    visited[node] = true;
    for (int[] neighbor : adj.get(node)) {
        if (!visited[neighbor[0]]) topologicalSort(neighbor[0], visited,
stack, adj);
    }
    stack.push(node);
```

k

```
}
```

PROBLEM:31

Dijkstra's Algorithm
Level: Hard

Java Code:

```
java
CopyEdit
class Pair {
    int node, dist;
    Pair(int d, int n) {
        dist = d;
        node = n;
    }
}

public int[] dijkstra(int V, List<List<List<Integer>>> adj, int src) {
    int[] dist = new int[V];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<Pair> pq = new PriorityQueue<>((a, b) -> a.dist - b.dist);
    pq.add(new Pair(0, src));

    while (!pq.isEmpty()) {
        Pair p = pq.poll();
        int u = p.node;
        for (List<Integer> nei : adj.get(u)) {
            int v = nei.get(0), wt = nei.get(1);
            if (dist[u] + wt < dist[v]) {
                dist[v] = dist[u] + wt;
                pq.add(new Pair(dist[v], v));
            }
        }
    }

    return dist;
}
```

PROBLEM:32

Why Priority Queue is used in Dijkstra's Algorithm
Level: Medium

Explanation:

- Priority queue helps efficiently get the node with the **minimum tentative distance**.
- Ensures that we always process the closest unvisited node.
- Time complexity improves from **$O(V^2)$** to **$O((V+E) \log V)$** with a priority queue (min-heap).

---

k

PROBLEM:33

Shortest Path in Binary Maze
Level: Medium

Problem Statement:
0 represents wall, 1 represents path. From source to destination, find the shortest path using BFS.

Java Code:

```
java
CopyEdit
public int shortestPath(int[][] grid, int[] src, int[] dest) {
    int n = grid.length, m = grid[0].length;
    if (src[0] == dest[0] && src[1] == dest[1]) return 0;
    int[][] dist = new int[n][m];
    for (int[] row : dist) Arrays.fill(row, Integer.MAX_VALUE);
    dist[src[0]][src[1]] = 0;

    Queue<int[]> q = new LinkedList<>();
    q.add(new int[]{src[0], src[1]});
    int[] dx = {0, 0, 1, -1};
    int[] dy = {1, -1, 0, 0};

    while (!q.isEmpty()) {
        int[] curr = q.poll();
        for (int i = 0; i < 4; i++) {
            int nx = curr[0] + dx[i], ny = curr[1] + dy[i];
            if (nx >= 0 && ny >= 0 && nx < n && ny < m && grid[nx][ny] == 1)
{
                if (dist[curr[0]][curr[1]] + 1 < dist[nx][ny]) {
                    dist[nx][ny] = dist[curr[0]][curr[1]] + 1;
                    q.add(new int[]{nx, ny});
                }
            }
        }
    }

    int ans = dist[dest[0]][dest[1]];
    return ans == Integer.MAX_VALUE ? -1 : ans;
}
```

PROBLEM:34

Path with Minimum Effort
Level: Medium

Problem Statement:
Given a 2D grid of heights, return the minimum effort path from top-left to bottom-right.
Effort = maximum absolute difference in heights between consecutive cells in the path.

Java Code:

```
java
CopyEdit
class Pair {
```

```
    int diff, x, y;
    Pair(int d, int i, int j) {
        diff = d;
        x = i;
        y = j;
    }
}

public int minimumEffortPath(int[][] heights) {
    int n = heights.length, m = heights[0].length;
    int[][] dist = new int[n][m];
    for (int[] row : dist) Arrays.fill(row, Integer.MAX_VALUE);
    dist[0][0] = 0;

    PriorityQueue<Pair> pq = new PriorityQueue<>((a, b) -> a.diff - b.diff);
    pq.add(new Pair(0, 0, 0));
    int[] dx = {0, 0, 1, -1}, dy = {1, -1, 0, 0};

    while (!pq.isEmpty()) {
        Pair curr = pq.poll();
        if (curr.x == n - 1 && curr.y == m - 1) return curr.diff;

        for (int i = 0; i < 4; i++) {
            int nx = curr.x + dx[i], ny = curr.y + dy[i];
            if (nx >= 0 && ny >= 0 && nx < n && ny < m) {
                int newEffort = Math.max(curr.diff,
Math.abs(heights[curr.x][curr.y] - heights[nx][ny]));
                if (newEffort < dist[nx][ny]) {
                    dist[nx][ny] = newEffort;
                    pq.add(new Pair(newEffort, nx, ny));
                }
            }
        }
    }
    return 0;
}
```

PROBLEM:35

Cheapest Flights Within K Stops
Level: Hard

Java Code:

```
java
CopyEdit
class Tuple {
    int stops, city, cost;
    Tuple(int c, int u, int s) {
        cost = c;
        city = u;
        stops = s;
    }
}

public int findCheapestPrice(int n, int[][] flights, int src, int dst, int
k) {
    List<List<int[]>> adj = new ArrayList<>();
```

```
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    for (int[] flight : flights) {
        adj.get(flight[0]).add(new int[]{flight[1], flight[2]});
    }

    Queue<Tuple> q = new LinkedList<>();
    q.add(new Tuple(0, src, 0));

    int[] costArr = new int[n];
    Arrays.fill(costArr, Integer.MAX_VALUE);
    costArr[src] = 0;

    while (!q.isEmpty()) {
        Tuple t = q.poll();
        if (t.stops > k) continue;
        for (int[] nei : adj.get(t.city)) {
            int nextCity = nei[0], price = nei[1];
            if (t.cost + price < costArr[nextCity]) {
                costArr[nextCity] = t.cost + price;
                q.add(new Tuple(t.cost + price, nextCity, t.stops + 1));
            }
        }
    }
    return costArr[dst] == Integer.MAX_VALUE ? -1 : costArr[dst];
}
```

PROBLEM:36

Bellman-Ford Algorithm
Level: Hard

Java Code:

```
java
CopyEdit
public int[] bellmanFord(int V, int[][] edges, int src) {
    int[] dist = new int[V];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    for (int i = 0; i < V - 1; i++) {
        for (int[] edge : edges) {
            int u = edge[0], v = edge[1], wt = edge[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + wt < dist[v]) {
                dist[v] = dist[u] + wt;
            }
        }
    }

    // Check for negative cycles
    for (int[] edge : edges) {
        int u = edge[0], v = edge[1], wt = edge[2];
        if (dist[u] != Integer.MAX_VALUE && dist[u] + wt < dist[v]) {
            return new int[]{-1}; // Negative weight cycle detected
        }
    }
    return dist;
}
```

k

PROBLEM:37

Floyd-Warshall Algorithm
Level: Hard

Java Code:

```
java
CopyEdit
public int[][] floydWarshall(int[][] graph, int V) {
    int[][] dist = new int[V][V];

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            dist[i][j] = graph[i][j];
        }
    }

    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] != Integer.MAX_VALUE && dist[k][j] !=
Integer.MAX_VALUE) {
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }
    }

    return dist;
}
```

PROBLEM:38

Find the City With the Smallest Number of Neighbors Within Threshold Distance
Level: Hard

Java Code:

```
java
CopyEdit
public int findTheCity(int n, int[][] edges, int distanceThreshold) {
    int[][] dist = new int[n][n];
    for (int[] row : dist) Arrays.fill(row, (int)1e9);
    for (int i = 0; i < n; i++) dist[i][i] = 0;

    for (int[] edge : edges) {
        dist[edge[0]][edge[1]] = edge[2];
        dist[edge[1]][edge[0]] = edge[2];
    }

    for (int k = 0; k < n; k++) {
```

```
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }

    int city = -1, minCount = n;
    for (int i = 0; i < n; i++) {
        int count = 0;
        for (int j = 0; j < n; j++) {
            if (i != j && dist[i][j] <= distanceThreshold) count++;
        }
        if (count <= minCount) {
            minCount = count;
            city = i;
        }
    }

    return city;
}
```

PROBLEM:39

Network Delay Time
Level: Medium

Java Code:

```
java
CopyEdit
public int networkDelayTime(int[][] times, int n, int k) {
    List<List<int[]>> graph = new ArrayList<>();
    for (int i = 0; i <= n; i++) graph.add(new ArrayList<>());

    for (int[] time : times)
        graph.get(time[0]).add(new int[]{time[1], time[2]});

    int[] dist = new int[n + 1];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[k] = 0;

    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    pq.offer(new int[]{k, 0});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int u = curr[0], time = curr[1];
        if (time > dist[u]) continue;

        for (int[] nei : graph.get(u)) {
            int v = nei[0], w = nei[1];
            if (time + w < dist[v]) {
                dist[v] = time + w;
                pq.offer(new int[]{v, dist[v]});
            }
        }
    }
```

```
    int max = 0;
    for (int i = 1; i <= n; i++) {
        if (dist[i] == Integer.MAX_VALUE) return -1;
        max = Math.max(max, dist[i]);
    }
    return max;
}
```

PROBLEM:40

Number of Ways to Arrive at Destination
Level: Medium

Java Code:

```
java
CopyEdit
public int countPaths(int n, int[][] roads) {
    List<List<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    for (int[] road : roads) {
        adj.get(road[0]).add(new int[]{road[1], road[2]});
        adj.get(road[1]).add(new int[]{road[0], road[2]});
    }

    long[] dist = new long[n];
    int[] ways = new int[n];
    Arrays.fill(dist, Long.MAX_VALUE);
    dist[0] = 0;
    ways[0] = 1;

    PriorityQueue<long[]> pq = new PriorityQueue<>((a, b) ->
Long.compare(a[0], b[0]));
    pq.add(new long[]{0, 0}); // time, node
    int mod = (int) 1e9 + 7;

    while (!pq.isEmpty()) {
        long[] curr = pq.poll();
        long time = curr[0];
        int node = (int) curr[1];

        for (int[] nei : adj.get(node)) {
            int next = nei[0], wt = nei[1];
            if (time + wt < dist[next]) {
                dist[next] = time + wt;
                ways[next] = ways[node];
                pq.add(new long[]{dist[next], next});
            } else if (time + wt == dist[next]) {
                ways[next] = (ways[next] + ways[node]) % mod;
            }
        }
    }
    return ways[n - 1];
}
```

PROBLEM:41

k

Minimum Steps to Reach End Using Multiplication and Mod
Level: Hard

Java Code:

```
java
CopyEdit
public int minimumMultiplications(int[] arr, int start, int end) {
    int mod = 100000;
    int[] dist = new int[mod];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[start] = 0;

    Queue<Integer> q = new LinkedList<>();
    q.add(start);

    while (!q.isEmpty()) {
        int node = q.poll();
        for (int num : arr) {
            int next = (node * num) % mod;
            if (dist[node] + 1 < dist[next]) {
                dist[next] = dist[node] + 1;
                if (next == end) return dist[next];
                q.add(next);
            }
        }
    }
    return -1;
}
```

PROBLEM:42

Bellman Ford Algorithm
Level: Hard

Java Code:

```
java
CopyEdit
class Edge {
    int u, v, wt;
    Edge(int u, int v, int wt) {
        this.u = u;
        this.v = v;
        this.wt = wt;
    }
}

public int[] bellmanFord(int V, ArrayList<Edge> edges, int S) {
    int[] dist = new int[V];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[S] = 0;

    for (int i = 0; i < V - 1; i++) {
        for (Edge e : edges) {
            if (dist[e.u] != Integer.MAX_VALUE && dist[e.u] + e.wt <
dist[e.v]) {
                dist[e.v] = dist[e.u] + e.wt;
```

```
            }
        }
    }

    // Optional negative cycle check
    for (Edge e : edges) {
        if (dist[e.u] != Integer.MAX_VALUE && dist[e.u] + e.wt < dist[e.v])
{
            return new int[]{-1}; // Negative cycle
        }
    }
    return dist;
}
```

PROBLEM:43

Floyd Warshall Algorithm
Level: Hard

Java Code:

```
java
CopyEdit
public void floydWarshall(int[][] dist) {
    int n = dist.length;
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] == Integer.MAX_VALUE || dist[k][j] ==
Integer.MAX_VALUE) continue;
                dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }
}
```

PROBLEM:44

Find the city with the smallest number of neighbors in a threshold distance
Level: Hard

Java Code:

```
java
CopyEdit
public int findTheCity(int n, int[][] edges, int distanceThreshold) {
    int[][] dist = new int[n][n];
    for (int[] row : dist) Arrays.fill(row, (int) 1e9);
    for (int i = 0; i < n; i++) dist[i][i] = 0;

    for (int[] edge : edges) {
        int u = edge[0], v = edge[1], w = edge[2];
        dist[u][v] = w;
        dist[v][u] = w;
    }
```

```
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);

    int minCount = n, ans = -1;
    for (int i = 0; i < n; i++) {
        int count = 0;
        for (int j = 0; j < n; j++) {
            if (i != j && dist[i][j] <= distanceThreshold) count++;
        }
        if (count <= minCount) {
            minCount = count;
            ans = i;
        }
    }
    return ans;
}
```

☑ PROBLEM: 44

🧠 Find the City with the Smallest Number of Neighbors in a Threshold Distance

Level: Hard

📝 Explanation:

We are given a graph and a distance threshold. For each city (node), we need to determine how many other cities are reachable within the threshold. The city with the **minimum number of such neighbors** is the answer. In case of a tie, we return the city with the **larger index**.

We solve this using the **Floyd-Warshall Algorithm** to compute shortest paths between all pairs of cities.

⏱ Time Complexity: `O(n^3)`

💾 Space Complexity: `O(n^2)`

👨‍💻 Java Code:

```
java
CopyEdit
public int findTheCity(int n, int[][] edges, int distanceThreshold) {
    int[][] dist = new int[n][n];
```

k

```
    // Initialize all distances to infinity
    for (int[] row : dist) Arrays.fill(row, (int) 1e9);

    // Distance to self is 0
    for (int i = 0; i < n; i++) dist[i][i] = 0;

    // Fill the direct edges
    for (int[] edge : edges) {
        int u = edge[0], v = edge[1], w = edge[2];
        dist[u][v] = w;
        dist[v][u] = w;
    }

    // Floyd-Warshall algorithm
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);

    int minCount = n, ans = -1;

    // Find the city with the least neighbors within the threshold
    for (int i = 0; i < n; i++) {
        int count = 0;
        for (int j = 0; j < n; j++) {
            if (i != j && dist[i][j] <= distanceThreshold) count++;
        }
        if (count <= minCount) {
            minCount = count;
            ans = i;
        }
    }

    return ans;
}
```

Let me know the next **problem number** you'd like in this format!

4o

You said:

continue with no emojis

ChatGPT said:

Certainly! Here's the next problem in the same clean format without emojis:

PROBLEM: 45

Minimum Spanning Tree

Level: Hard

k

Explanation:

A **Minimum Spanning Tree (MST)** of a weighted undirected graph is a subset of the edges that connects all vertices together, without any cycles and with the minimum possible total edge weight.

We can use **Kruskal's Algorithm** along with the **Disjoint Set Union (DSU)** data structure to implement this efficiently.

Time Complexity: O(E log E + E * α(N))

Space Complexity: O(N)

Java Code:

```
java
CopyEdit
class DisjointSet {
    int[] parent, rank;

    public DisjointSet(int n) {
        parent = new int[n];
        rank = new int[n];
        for(int i = 0; i < n; i++) parent[i] = i;
    }

    public int find(int x) {
        if(parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }

    public void union(int u, int v) {
        int pu = find(u);
        int pv = find(v);
        if(pu == pv) return;

        if(rank[pu] < rank[pv]) {
            parent[pu] = pv;
        } else if(rank[pv] < rank[pu]) {
            parent[pv] = pu;
        } else {
            parent[pu] = pv;
            rank[pv]++;
        }
    }
}

public int minimumSpanningTree(int n, int[][] edges) {
    Arrays.sort(edges, (a, b) -> Integer.compare(a[2], b[2]));
    DisjointSet dsu = new DisjointSet(n);

k
```

```
    int mstWeight = 0;

    for(int[] edge : edges) {
        int u = edge[0], v = edge[1], wt = edge[2];
        if(dsu.find(u) != dsu.find(v)) {
            dsu.union(u, v);
            mstWeight += wt;
        }
    }

    return mstWeight;
}
```

PROBLEM: 46

Prim's Algorithm

Level: Hard

Explanation:

Prim’s algorithm builds the MST by picking one node and adding the minimum weight edge that connects a new node to the growing MST using a **PriorityQueue** (min-heap).

Time Complexity: O(E log V)

Space Complexity: O(V + E)

Java Code:

```
java
CopyEdit
class Pair {
    int node, weight;
    Pair(int node, int weight) {
        this.node = node;
        this.weight = weight;
    }
}

public int primMST(int n, List<List<Pair>> adj) {
    boolean[] visited = new boolean[n];
    PriorityQueue<Pair> pq = new PriorityQueue<>((a, b) -> a.weight -
b.weight);
    pq.add(new Pair(0, 0));
    int mstWeight = 0;

    while(!pq.isEmpty()) {
        Pair current = pq.poll();
```

```
        int node = current.node, weight = current.weight;
        if(visited[node]) continue;
        visited[node] = true;
        mstWeight += weight;

        for(Pair neighbor : adj.get(node)) {
            if(!visited[neighbor.node]) {
                pq.add(new Pair(neighbor.node, neighbor.weight));
            }
        }
    }

    return mstWeight;
}
```

PROBLEM: 47

Disjoint Set [Union by Rank]

Level: Hard

Explanation:

Disjoint Set Union (DSU) helps manage components and their connections. `Union by Rank` keeps tree shallow, improving efficiency.

Time Complexity: O(α(N)) per operation

Space Complexity: O(N)

Java Code:

```
java
CopyEdit
class DisjointSet {
    int[] parent, rank;

    public DisjointSet(int n) {
        parent = new int[n];
        rank = new int[n];
        for(int i = 0; i < n; i++) parent[i] = i;
    }

    public int find(int x) {
        if(parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
```

```
    }

    public void union(int u, int v) {
        int pu = find(u);
        int pv = find(v);
        if(pu == pv) return;

        if(rank[pu] < rank[pv]) {
            parent[pu] = pv;
        } else if(rank[pv] < rank[pu]) {
            parent[pv] = pu;
        } else {
            parent[pu] = pv;
            rank[pv]++;
        }
    }
}
```

PROBLEM: 48

Disjoint Set [Union by Size]

Level: Hard

Explanation:

Same as union by rank, but instead of rank, we use the **size** of the trees to merge smaller into larger.

Time Complexity: O(α(N)) per operation

Space Complexity: O(N)

Java Code:

```
java
CopyEdit
class DisjointSet {
    int[] parent, size;

    public DisjointSet(int n) {
        parent = new int[n];
        size = new int[n];
        for(int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }
```

k

```
    public int find(int x) {
        if(parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }

    public void union(int u, int v) {
        int pu = find(u);
        int pv = find(v);
        if(pu == pv) return;

        if(size[pu] < size[pv]) {
            parent[pu] = pv;
            size[pv] += size[pu];
        } else {
            parent[pv] = pu;
            size[pu] += size[pv];
        }
    }
}
```

PROBLEM: 49

Kruskal's Algorithm

Level: Hard

Explanation:

Kruskal’s Algorithm builds MST by sorting all edges and picking the smallest edge that doesn’t form a cycle using DSU.

Time Complexity: O(E log E + E·α(N))

Space Complexity: O(N)

Java Code:

```
java
CopyEdit
class DisjointSet {
    int[] parent, rank;
    public DisjointSet(int n) {
        parent = new int[n];
        rank = new int[n];
        for(int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int u) {
```

```
        if (parent[u] != u) parent[u] = find(parent[u]);
        return parent[u];
    }
    void union(int u, int v) {
        int pu = find(u), pv = find(v);
        if (pu == pv) return;
        if (rank[pu] < rank[pv]) parent[pu] = pv;
        else if (rank[pv] < rank[pu]) parent[pv] = pu;
        else {
            parent[pv] = pu;
            rank[pu]++;
        }
    }
}

public int kruskalMST(int n, int[][] edges) {
    Arrays.sort(edges, (a, b) -> a[2] - b[2]);
    DisjointSet ds = new DisjointSet(n);
    int mstWeight = 0;
    for (int[] edge : edges) {
        int u = edge[0], v = edge[1], wt = edge[2];
        if (ds.find(u) != ds.find(v)) {
            ds.union(u, v);
            mstWeight += wt;
        }
    }
    return mstWeight;
}
```

PROBLEM: 50

Number of operations to make network connected

Level: Medium

Explanation:

We use DSU to count connected components. If we have enough extra edges (`total edges >= n - 1`), we can connect the graph.

Time Complexity: O(E·α(N))

Space Complexity: O(N)

Java Code:

`java`

k

```
CopyEdit
class DisjointSet {
    int[] parent;
    public DisjointSet(int n) {
        parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (x != parent[x]) parent[x] = find(parent[x]);
        return parent[x];
    }
    void union(int x, int y) {
        parent[find(x)] = find(y);
    }
}

public int makeConnected(int n, int[][] connections) {
    if (connections.length < n - 1) return -1;

    DisjointSet ds = new DisjointSet(n);
    int components = n;
    for (int[] conn : connections) {
        if (ds.find(conn[0]) != ds.find(conn[1])) {
            ds.union(conn[0], conn[1]);
            components--;
        }
    }
    return components - 1;
}
```

PROBLEM: 51

Most stones removed with same rows or columns

Level: Medium

Explanation:

Treat each stone as a node and connect stones that share row/column using DSU. The answer is total stones - number of connected components.

Time Complexity: O(N·α(N))

Space Complexity: O(N)

Java Code:

```
java
```

k

```
CopyEdit
class DisjointSet {
    Map<Integer, Integer> parent = new HashMap<>();
    public int find(int x) {
        parent.putIfAbsent(x, x);
        if (x != parent.get(x)) parent.put(x, find(parent.get(x)));
        return parent.get(x);
    }
    public void union(int x, int y) {
        parent.put(find(x), find(y));
    }
}

public int removeStones(int[][] stones) {
    DisjointSet ds = new DisjointSet();
    for (int[] stone : stones) {
        ds.union(stone[0], ~(stone[1])); // encode column as negative
    }
    Set<Integer> unique = new HashSet<>();
    for (int[] stone : stones) {
        unique.add(ds.find(stone[0]));
    }
    return stones.length - unique.size();
}
```

PROBLEM: 52

Accounts Merge

Level: Hard

Explanation:

Use DSU to group accounts based on shared emails. Then collect emails for each parent and format the result.

Time Complexity: O(N·α(N))

Space Complexity: O(N)

Java Code:

```
java
CopyEdit
class DisjointSet {
    int[] parent;
    public DisjointSet(int n) {
        parent = new int[n];
```

```
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (x != parent[x]) parent[x] = find(parent[x]);
        return parent[x];
    }
    void union(int x, int y) {
        parent[find(x)] = find(y);
    }
}

public List<List<String>> accountsMerge(List<List<String>> accounts) {
    int n = accounts.size();
    DisjointSet ds = new DisjointSet(n);
    Map<String, Integer> emailToIndex = new HashMap<>();

    for (int i = 0; i < n; i++) {
        for (int j = 1; j < accounts.get(i).size(); j++) {
            String email = accounts.get(i).get(j);
            if (emailToIndex.containsKey(email)) {
                ds.union(i, emailToIndex.get(email));
            } else {
                emailToIndex.put(email, i);
            }
        }
    }

    Map<Integer, Set<String>> indexToEmails = new HashMap<>();
    for (Map.Entry<String, Integer> entry : emailToIndex.entrySet()) {
        int parent = ds.find(entry.getValue());
        indexToEmails.computeIfAbsent(parent, x -> new
TreeSet<>()).add(entry.getKey());
    }

    List<List<String>> result = new ArrayList<>();
    for (Map.Entry<Integer, Set<String>> entry : indexToEmails.entrySet())
{
        List<String> merged = new ArrayList<>();
        merged.add(accounts.get(entry.getKey()).get(0));
        merged.addAll(entry.getValue());
        result.add(merged);
    }
    return result;
}
```

PROBLEM: 53

Number of Islands II

Level: Hard

Explanation:

Use DSU to dynamically track connected land components as we add new land cells.

Time Complexity: O(k·α(n))

Space Complexity: O(n)

Java Code:

```
java
CopyEdit
class DisjointSet {
    int[] parent, rank;
    public DisjointSet(int n) {
        parent = new int[n];
        rank = new int[n];
        Arrays.fill(parent, -1);
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    void union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return;
        if (rank[px] < rank[py]) parent[px] = py;
        else if (rank[px] > rank[py]) parent[py] = px;
        else {
            parent[py] = px;
            rank[px]++;
        }
    }
}

public List<Integer> numIslands2(int m, int n, int[][] positions) {
    DisjointSet ds = new DisjointSet(m * n);
    int[][] grid = new int[m][n];
    List<Integer> result = new ArrayList<>();
    int count = 0;

    int[][] dirs = {{0,1}, {1,0}, {0,-1}, {-1,0}};
    for (int[] pos : positions) {
        int x = pos[0], y = pos[1];
        if (grid[x][y] == 1) {
            result.add(count);
            continue;
        }
        grid[x][y] = 1;
        count++;
        int id = x * n + y;
        ds.parent[id] = id;

        for (int[] d : dirs) {
            int nx = x + d[0], ny = y + d[1];
            int nid = nx * n + ny;
            if (nx >= 0 && ny >= 0 && nx < m && ny < n && grid[nx][ny] == 1)
{


k
```

```
                int p1 = ds.find(id), p2 = ds.find(nid);
                if (p1 != p2) {
                    ds.union(p1, p2);
                    count--;
                }
            }
        }
        result.add(count);
    }
    return result;
}
```

PROBLEM: 54

Making a Large Island

Level: Hard

Explanation:

Label each island and count their size. Then for every 0, try flipping it and sum sizes of adjacent unique islands.

Time Complexity: O(n²)

Space Complexity: O(n²)

Java Code:

```
java
CopyEdit
public int largestIsland(int[][] grid) {
    int n = grid.length;
    int[][] dirs = {{0,1}, {1,0}, {0,-1}, {-1,0}};
    Map<Integer, Integer> areaMap = new HashMap<>();
    int idx = 2;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == 1) {
                areaMap.put(idx, dfs(grid, i, j, idx, dirs));
                idx++;
            }
        }
    }

    int max = areaMap.values().stream().max(Integer::compare).orElse(0);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
```

```
            if (grid[i][j] == 0) {
                Set<Integer> seen = new HashSet<>();
                int area = 1;
                for (int[] d : dirs) {
                    int ni = i + d[0], nj = j + d[1];
                    if (ni >= 0 && nj >= 0 && ni < n && nj < n &&
grid[ni][nj] > 1) {
                        if (seen.add(grid[ni][nj])) {
                            area += areaMap.get(grid[ni][nj]);
                        }
                    }
                }
                max = Math.max(max, area);
            }
        }
    }
    return max == 0 ? n * n : max;
}

private int dfs(int[][] grid, int i, int j, int idx, int[][] dirs) {
    if (i < 0 || j < 0 || i >= grid.length || j >= grid.length ||
grid[i][j] != 1) return 0;
    grid[i][j] = idx;
    int area = 1;
    for (int[] d : dirs) area += dfs(grid, i + d[0], j + d[1], idx, dirs);
    return area;
}
```

PROBLEM: 55

Swim in Rising Water

Level: Hard

Explanation:

Use Dijkstra's Algorithm to find the path where the highest elevation along the path is minimized.

Time Complexity: O(n² log n)

Space Complexity: O(n²)

Java Code:

```
java
CopyEdit
public int swimInWater(int[][] grid) {
```

k

```
    int n = grid.length;
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a
-> a[0]));
    boolean[][] visited = new boolean[n][n];
    pq.offer(new int[]{grid[0][0], 0, 0});
    int[][] dirs = {{0,1}, {1,0}, {0,-1}, {-1,0}};

    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int time = cur[0], x = cur[1], y = cur[2];
        if (x == n - 1 && y == n - 1) return time;
        if (visited[x][y]) continue;
        visited[x][y] = true;

        for (int[] d : dirs) {
            int nx = x + d[0], ny = y + d[1];
            if (nx >= 0 && ny >= 0 && nx < n && ny < n && !visited[nx][ny])
{
                pq.offer(new int[]{Math.max(time, grid[nx][ny]), nx, ny});
            }
        }
    }
    return -1;
}
```

***Congratulations on completing the PDF created by** SYNTAX ERROR**, also known as Abhishek Rathor! Your dedication and perseverance are truly commendable. For more insights and updates, feel free to connect on Instagram at** SYNTAX ERROR.*

k